Python for Coding Interviews

NeetCode 150 and the original Blind 75

Rebuild the idea. Write the code. Adapt the rule.

Use this guide to recover skills you have learned before. The Python section gives the operations used in these problems. The technique sections explain the shared reasoning and show small, executable examples. The final map covers all 151 distinct problems: NeetCode 150 and Combination Sum IV from the original Blind 75. Each row gives the change needed for that problem and links to its technique.

Read a technique when you cannot explain its rule or write its loop. Then close the guide and rebuild the example. For a familiar problem, use the final map as a recall prompt.

Contents

1 Find the next useful step **2**
2 Python operations to recall **2**
3 Hashing: ask an exact question **7**
4 Prefix and suffix: summarize each side **8**
5 Two pointers: justify what you discard **9**
6 Sliding windows: maintain one contiguous range **10**
7 Stacks: keep unfinished work **11**
8 Binary search: find a boundary **12**
9 Linked lists: preserve links before changing them **14**
10 Trees: decide what travels up or down **17**
11 Tries: share work across prefixes **18**
12 Heaps: keep the next useful candidate **19**
13 Backtracking: choose, explore, undo **21**
14 Graph traversal: choose the state and the frontier **23**
15 Graph structure: readiness and connectivity **24**
16 Weighted paths and using every edge **26**
17 DP: keep only the facts that change the future **28**
18 DP on totals: distinguish reuse and order **29**
19 DP on two sequences, grids, and intervals **31**
20 Greedy: prove that alternatives can be discarded **33**
21 Intervals: choose a sweep order **34**
22 Matrices and number operations **36**
23 Bits: operate on a fixed representation **38**
24 Recall and transfer in a few minutes **39**
25 Problem map: all 151 problems **40**
Sources and verification **52**

1 Find the next useful step

1. State what counts as a valid answer. Check order, duplicates, reuse, and whether you must change the input.
2. Describe a direct solution. Find the search, repeated calculation, or repeated state that makes it expensive.
3. Choose the smallest summary that removes that work. State exactly what each variable or table entry means.
4. Explain why an update preserves that meaning. Then write the loop and test a small case.

For a pair with a target sum, a direct solution searches for a partner for each value. Once `x` is fixed, the partner is exactly `target - x`. A dictionary can answer that question. If the output changes from indexes to a count of pairs, store counts instead of indexes. The reusable idea is the question the dictionary answers.

Repeated work	Idea to test
Search earlier items for one exact value	Store membership, indexes, or counts (§3).
Recompute a summary on both sides	Build prefix and suffix information (§4).
Try every pair in sorted input	Prove that moving an end discards impossible pairs (§5).
Rebuild overlapping contiguous ranges	Add the new item and remove the old item (§6).
Search ahead for the first greater value	Keep unresolved candidates in a stack (§7).
Try every possible answer	Test whether feasibility changes only once (§8).
Repeatedly select the next smallest item	Keep candidates in a heap (§12).
Explore choices with the same future	Define the future state, then cache it (§17).
Find the fewest equal-cost moves	Model states and moves as a graph. Use BFS (§14).
Decide what can be done next	Track unmet prerequisites (§15).

Use constraints to reject work. About 10^5 items usually calls for a linear or sorting-based solution. A few thousand can allow a quadratic loop. A choice set near 20 can allow subset search. These are estimates, not time guarantees. Count states, transitions, copying, and output size. An explicit logarithmic-time or constant-space requirement is part of the task.

Test the assumption when the problem changes. Negative numbers can break a sum-based sliding window. Reuse changes a combination search. Weights change BFS. A move limit adds state. Duplicates change comparison rules. Keep the technique only if its correctness argument still holds.

2 Python operations to recall

Examples use Python 3.10+ features and work on Python 3.13. Heap examples use negation for a max-heap. In a code block, `# ->` introduces the expected result.

2.1 Loops, indexes, and conditions

Iterate values directly. Use `enumerate` when you also need an index. `range(start, stop, step)` excludes `stop`. A slice excludes its right end.

```
nums = [4, 7, 2]
list(enumerate(nums))                  # -> [(0, 4), (1, 7), (2, 2)]
list(range(2, -1, -1))                 # -> [2, 1, 0]
nums[1:3], nums[-1], nums[::-1]         # -> ([7, 2], 2, [2, 7, 4])
list(zip([1, 2], [10, 20]))            # -> [(1, 10), (2, 20)]
sum(x * x for x in nums)               # -> 69
all(x > 0 for x in nums)               # -> True
any(x % 2 == 0 for x in nums)          # -> True
[x for x in nums if x > 3]            # -> [4, 7]
[(r, c) for r in range(2) for c in range(2)]
# -> [(0, 0), (0, 1), (1, 0), (1, 1)]
value = 5 if nums else 0
value                                  # -> 5
```

Empty containers, zero, `None`, and `False` are false in a condition. Use `is None` when zero is valid. Use `==` for equal values and `is` for the same object. `and` and `or` short-circuit and return an operand.

```
stack = []
bool(stack), bool(0), bool(None)        # -> (False, False, False)
bool(stack and stack[-1] == 3)          # -> False
x = 0
x is None                              # -> False
0 <= x < 3                             # -> True
```

In a `while` loop, each path must move a pointer, remove work, or return. `break` leaves the loop; `continue` starts its next iteration. Check bounds before indexing. A negative index can silently read the last item.

2.2 Lists, tuples, and copies

Use a list for a stack or indexed table. `append` adds one object; `extend` adds its elements. `pop()` returns the last element. `pop(0)` shifts the remaining list.

```
a = [1]
a.append([2, 3])
a                                      # -> [1, [2, 3]]
a.extend([4, 5])
a.pop()                                # -> 5
a                                      # -> [1, [2, 3], 4]
r, c = (2, 3)
r, c = c, r                            # read the right side first
(r, c), (4,)                          # -> ((3, 2), (4,))
```

Assignment shares an object. A shallow copy copies only the outer list. For a numeric grid, copy each row. Allocate each row separately.

```
grid = [[0] * 2 for _ in range(2)]
alias = grid
outer = grid.copy()
copied = [row[:] for row in grid]
grid[0][0] = 7
alias, outer, copied
# -> ([[7, 0], [0, 0]], [[7, 0], [0, 0]], [[0, 0], [0, 0]])
bad = [[0] * 2] * 2
bad[0][0] = 9
bad                                  # -> [[9, 0], [9, 0]]
```

Save a backtracking path with `path.copy()`. Use `copy.deepcopy` if deeper mutable objects also need independent copies. For node graphs, use an original-to-copy map (§9).

2.3 Dictionaries, sets, and counting

Dictionary membership tests keys. `get(key, default)` reads without inserting. Keys must be hashable: numbers, strings, and tuples of hashable items work; lists do not.

```
seen = {4: 0}
4 in seen, seen.get(9, -1)           # -> (True, -1)
seen[7] = 1
list(seen.items()), list(seen.values()) # -> ([(4, 0), (7, 1)], [0, 1])
del seen[4]
seen                                 # -> {7: 1}
cells = {(0, 1)}
cells.add((2, 3))
cells.discard((9, 9))                # absent is allowed
cells.remove((0, 1))                # absent would raise KeyError
cells                                # -> {(2, 3)}
{1, 2} & {2, 3}, {1, 2} | {2, 3}    # -> ({2}, {1, 2, 3})
{1, 2} - {2, 3}                      # -> {1}
```

```
from collections import Counter, defaultdict
count = Counter("aba")
count["a"], count["z"]               # -> (2, 0)
Counter("aab") == Counter("aba")     # -> True
groups = defaultdict(list)           # factory: no parentheses
for word in ["eat", "tea", "tan"]:
    groups["".join(sorted(word))].append(word)
list(groups.values())               # -> [['eat', 'tea'], ['tan']]
```

`defaultdict` inserts a default on `d[key]`, but not on `d.get(key)`. Do not add or delete keys while iterating a dictionary or set. Iterate a snapshot such as `list(d)` if needed.

2.4 Strings and ordering

Strings are immutable. Collect output pieces, then join them. `sorted(s)` returns a character list. Tuples compare from left to right.

```
s = "A man, 12!"
"".join(ch.lower() for ch in s if ch.isalnum()) # -> 'aman12'
"a b  c".split(), "a,b,,c".split(",") # -> (['a', 'b', 'c'], ['a', 'b', '', 'c'])
"12#abc".find("#"), int("12"), str(12) # -> (2, 12, '12')
ord("c") - ord("a"), chr(ord("a") + 2) # -> (2, 'c')
"ab" in "zabc", "abc".startswith("ab") # -> (True, True)
f"{3}#{'cat'}"                        # -> '3#cat'
sorted([3, 1, 2], reverse=True)         # -> [3, 2, 1]
items = [(1, 2), (0, 8), (1, 5)]
sorted(items, key=lambda p: (p[0], -p[1]))
# -> [(0, 8), (1, 5), (1, 2)]
min(items, key=lambda p: p[1])          # -> (1, 2)
(1, 8) < (2, 0)                       # -> True
nums = [3, 1, 2]
result = nums.sort()
nums, result                           # -> ([1, 2, 3], None)
```

`sorted` makes a new list. `sort` and `reverse` change the list and return `None`. Keep original indexes with `enumerate` if you must restore output order.

2.5 Queues, heaps, and bisection

Use `deque` for FIFO. A heap returns its smallest item; its whole list is not sorted. Negate numeric priorities for largest-first selection.

```
from collections import deque
from heapq import heapify, heappush, heappop
q = deque([1, 2])
q.append(3)
q.popleft(), q[-1]                    # -> (1, 3)
h = [5, 1, 3]
heapify(h)                             # O(n), changes h
h[0]                                   # -> 1
heappush(h, 2)
[heappop(h) for _ in range(4)]         # -> [1, 2, 3, 5]
h = [-5, -1, -3]
heapify(h)
-heappop(h)                           # -> 5
```

Heap tuples compare each field. Use `(priority, unique_id, node)` when nodes cannot be ordered.

```
from bisect import bisect_left, bisect_right
a = [1, 2, 2, 4]
bisect_left(a, 2), bisect_right(a, 2)  # -> (1, 3)
bisect_left(a, 3), bisect_right(a, 9)  # -> (3, 4)
i = bisect_right(a, 3) - 1             # last value <= 3
a[i] if i >= 0 else None              # -> 2
```

`bisect_left` finds the first value at least `x`; `bisect_right` finds the first value greater than `x`. Either can return `len(a)`. Search is logarithmic; insertion in the middle is still linear.

2.6 Helpers, scope, caching, and classes

A missing return gives `None`. A helper can read outer names and change outer containers. Use `nonlocal` when it reassigns an outer variable.

```
def count_positive(nums):
    total, found = 0, []
    def visit(x):
        nonlocal total
        if x > 0:
            total += 1
            found.append(x)           # append needs no nonlocal
    for x in nums:
        visit(x)
    return total, found
count_positive([-1, 2, 3])            # -> (2, [2, 3])

from functools import cache
def stairs(n):
    @cache
    def ways(i):                      # fresh cache for each stairs call
        if i <= 1:
            return 1
        return ways(i - 1) + ways(i - 2)
    return ways(n)
stairs(4)                             # -> 5
```

Cache keys must be hashable. Include every value that changes the future answer. Do not change hidden state on which a cached answer depends. A cache does not remove recursion depth. Use a table, stack, or queue for deep chains.

```
class Node:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
head = Node(3, Node(5))
head.val, head.next.val               # -> (3, 5)
```

`self` is the current instance. Put per-instance containers in `__init__`. For `class Solution`, the judge calls your method with its supplied arguments. `List[int]` and `list[int]` are type hints; they do not convert or check values. Avoid mutable defaults such as `path=[]`. Allocate the list inside the function.

2.7 Arithmetic and costs

Python integers grow as needed. `/` produces a float; `//` rounds down. For positive integers, use an integer formula for ceiling division.

```
7 // 3, 7 % 3, divmod(7, 3)          # -> (2, 1, (2, 1))
-7 // 3, -7 % 3                      # -> (-3, 2)
(11 + 4 - 1) // 4                    # -> 3
float("inf") > 10**100               # -> True
```

To truncate division toward zero, divide magnitudes with `//`, then apply the sign. For Reverse Polish Notation, $-7/3$ must give -2. Avoid float conversion.

Operation	Cost
List index, assignment, end pop	$O(1)$.
List append	Amortized $O(1)$.
Dictionary/set lookup, insert, delete	Average $O(1)$, plus key construction and hashing.

Operation	Cost
`x in list`, `index`, `min`, `max`, `sum`	$O(n)$.
Copy or slice of k elements	$O(k)$ time and space.
Sort	$O(n \log n)$ worst-case comparisons.
Heap push/pop; heap construction	$O(\log n)$; $O(n)$.
Deque operations at either end	$O(1)$.

Repeated slicing, concatenation, or `max(count.values())` can add work. The last operation is cheap for a fixed alphabet of 26 letters, but not for a growing set of keys.

3 Hashing: ask an exact question

Use a set for "did it occur?", a dictionary for "where?", and counts for "how many?". A large value range is fine: store only keys that occur.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
two_sum([3, 3], 6)                    # -> [0, 1]
```

Before index `i`, the map contains only earlier items. Look up before insertion so that an item cannot pair with itself. For a pair count, add `count[target - x]` to the answer, then increase `count[x]`. For Contains Duplicate, only membership matters. For Valid Anagram, compare two `Counter` objects.

Group by what stays the same. Anagrams have the same sorted characters. The key is the sorted word joined back into a string; see the Python grouping example. This costs $O(L \log L)$ per word of length L. If a linear bound is required, count the fixed alphabet and use the count tuple as the key. For Sudoku, a digit must be new to its row, column, and box. The box key is `(r // 3, c // 3)`.

Use a bounded property as an index. A frequency is at most the input length. This gives a direct way to return the top `k` frequencies without sorting all values.

```
from collections import Counter
def top_frequent(nums, k):
    buckets = [[] for _ in range(len(nums) + 1)]
    for value, count in Counter(nums).items():
        buckets[count].append(value)
    out = []
    for count in range(len(nums), 0, -1):
        out.extend(buckets[count])
        if len(out) >= k:
            return out[:k]
sorted(top_frequent([1, 1, 1, 2, 2, 3], 2)) # -> [1, 2]
```

For Longest Consecutive Sequence, put values in a set. Start a run only when `x - 1` is absent. Then increment while the next value exists. Iterate distinct values. Each run is scanned once in total.

Cost: Hash scans take average $O(n)$ time and $O(n)$ space..

For a new problem. Write the exact lookup key before choosing a hash table. To group equivalent

objects, remove only the irrelevant detail. If you need an inequality or nearest value, a plain hash lookup is insufficient; test sorting or binary search.

3.1 Recover boundaries with a length header

Encode and Decode Strings is a parsing problem. A delimiter can occur inside the data. Write the length, a marker, and the payload. Search for the marker only in the header; jump over the payload.

```
def encode(words):
    return "".join(f"{len(w)}#{w}" for w in words)
def decode(data):
    out, i = [], 0
    while i < len(data):
        j = data.index("#", i)
        size = int(data[i:j])
        i = j + 1
        out.append(data[i:i + size])
        i += size
    return out
decode(encode(["", "2#x", "hello"]))   # -> ['', '2#x', 'hello']
decode(encode([]))                     # -> []
```

The decoder assumes valid encoder output. The same Python character count is used on both sides. For a byte protocol, define lengths in bytes instead.

4 Prefix and suffix: summarize each side

Ask what a position needs from its left and right. Compute those summaries once. For Product of Array Except Self, multiplying the two sides avoids division and handles zeros.

```
def product_except_self(nums):
    out = [1] * len(nums)
    product = 1
    for i, x in enumerate(nums):
        out[i] = product               # strictly left of i
        product *= x
    product = 1
    for i in range(len(nums) - 1, -1, -1):
        out[i] *= product              # strictly right of i
        product *= nums[i]
    return out
product_except_self([1, 2, 3, 4])      # -> [24, 12, 8, 6]
product_except_self([0, 2, 0])         # -> [0, 0, 0]
```

For Trapping Rain Water, precompute the maximum height from each end. At index `i`, water is `min(left_max[i], right_max[i]) - height[i]`. Including `height[i]` in each maximum makes this nonnegative. For `[3, 0, 2, 0, 4]`, the water depths are `[0, 3, 1, 3, 0]`. This clear $O(n)$-space version is the default; two pointers can reduce the storage if required.

Transfer to range queries. Use an empty prefix so that every range uses one formula.

```
nums = [4, -2, 5]
prefix = [0]
for x in nums:
    prefix.append(prefix[-1] + x)
prefix[3] - prefix[1]               # -> 3
```

Here `prefix[i]` summarizes `nums[:i]`. The prefix ending at `right` includes the unwanted part before `left`. Subtract that earlier prefix to leave only the sum on `[left, right)`: `prefix[right] - prefix[left]`. If a new task asks for a count of exact-sum ranges, rearrange the equation and count earlier prefixes with hashing.

Cost: Two passes take $O(n)$ time. Product Except Self uses $O(1)$ extra space beyond its output..

For a new problem. Choose the summary from the operation: sum, product, minimum, maximum, or last occurrence. Only use subtraction to remove a prefix when the operation supports it; prefix maxima cannot be subtracted.

5 Two pointers: justify what you discard

In a sorted pair search, the smallest and largest remaining values bound all other choices. If their sum is too small, no pair using the left value can work. If too large, no pair using the right value can work.

```
def sorted_pair(a, target):
    left, right = 0, len(a) - 1
    while left < right:
        total = a[left] + a[right]
        if total < target:
            left += 1
        elif total > target:
            right -= 1
        else:
            return [left + 1, right + 1]  # required 1-based indexes
    return []
sorted_pair([2, 7, 11, 15], 9)        # -> [1, 2]
```

For 3Sum, sort, fix one anchor, then search its suffix for the opposite sum. Skip equal anchors. After a match, move both pointers and skip equal values at each end. This avoids duplicate triples while still allowing repeated values at different indexes. The sort costs $O(n \log n)$; the pair scans cost $O(n^2)$.

For Container With Most Water, record `min(height[left], height[right]) * (right - left)`. Move the shorter wall. Keeping it and moving the taller wall can only reduce width without improving the limiting height. For `[1, 8, 6, 2, 5, 4, 8, 3, 7]`, the best pair has height 7 and width 7, giving 49.

For Valid Palindrome, keep lowercase alphanumeric characters and compare with the reverse. If constant extra space is required, use two pointers that skip non-alphanumeric characters.

Cost: Each pointer moves in one direction, so an ordinary two-pointer pass takes $O(n)$ time..

For a new problem. Explain why the discarded endpoint cannot be part of a better answer among the remaining choices. If moving either end can lose a valid answer without such a proof, choose another method.

6 Sliding windows: maintain one contiguous range

Use a window when neighboring ranges share almost all their work. Choose the stored counts or candidates, then decide when the left end must move.

6.1 Longest valid: shrink while invalid

For a substring with no repeats, a set represents the current window.

```
def longest_unique(s):
    seen, left, best = set(), 0, 0
    for right, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[left])
            left += 1
        seen.add(ch)
        best = max(best, right - left + 1)
    return best
longest_unique("pwwkew")              # -> 3
longest_unique("")                    # -> 0
```

For Character Replacement, store letter counts. A window needs length − largest count replacements. Shrink while that exceeds `k`. Recompute the largest count over the 26 letters. This keeps the rule exact and avoids the more subtle stale-maximum optimization. For `"AABABBA"` with `k=1`, the answer is 4.

6.2 Shortest sufficient: shrink while valid

For Minimum Window Substring, track the number of required character copies still missing. Negative counts mean surplus copies. Save indexes during the scan, then slice once.

```
from collections import Counter
def min_window(s, t):
    if not t:
        return ""
    need, missing = Counter(t), len(t)
    left, best_start, best_size = 0, 0, len(s) + 1
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            if right - left + 1 < best_size:
                best_start, best_size = left, right - left + 1
            ch = s[left]
            need[ch] += 1
            if need[ch] > 0:
                missing += 1
            left += 1
    return "" if best_size > len(s) else s[best_start:best_start + best_size]
min_window("ADOBECODEBANC", "ABC")    # -> 'BANC'
min_window("a", "aa")                 # -> ''
```

For Permutation in String, the length is fixed at `len(s1)`. Add the new character, remove the outgoing character, and compare 26-element count arrays. Equal counts mean a permutation. For `"ab"` in

"eidbaooo", the matching window is "ba".

6.3 Keep only useful maximum candidates

A newer value that is at least as large makes an older candidate useless: it is larger and expires later. Store indexes in a deque, with decreasing values.

```
from collections import deque
def window_max(nums, k):
    q, out = deque(), []
    for i, x in enumerate(nums):
        while q and q[0] <= i - k:
            q.popleft()              # expired
        while q and nums[q[-1]] <= x:
            q.pop()                  # dominated
        q.append(i)
        if i >= k - 1:
            out.append(nums[q[0]])
    return out
window_max([1, 3, -1, -3, 5, 3, 6, 7], 3) # -> [3, 3, 5, 5, 6, 7]
```

Cost: Each item enters and leaves at most once: $O(n)$ time, apart from fixed-alphabet comparisons..

For a new problem. Check that validity gives a safe direction for the left pointer. With negative values, removing a left value can increase a sum; a usual sum-limit window may fail. For a fixed length, identify an updateable summary. For an extremum, test whether newer candidates dominate older ones.

7 Stacks: keep unfinished work

Use a stack when the most recent unfinished item must be handled first. For brackets, push openers; a closer must match the top. Finish with an empty stack. For reverse Polish notation, pop the right operand first, then the left.

```
def trunc_div(a, b):
    magnitude = abs(a) // abs(b)
    return -magnitude if (a < 0) != (b < 0) else magnitude
trunc_div(-7, 3), trunc_div(7, -3)    # -> (-2, -2)
stack = []                            # Min Stack stores (value, min so far)
for x in [3, 1, 2]:
    stack.append((x, min(x, stack[-1][1]) if stack else x))
stack[-1]                             # -> (2, 1)
```

7.1 Monotonic stack: resolve waiting items

For Daily Temperatures, each index waits for its first strictly warmer day.

```python
def warmer_days(temperatures):
    waiting, out = [], [0] * len(temperatures)
    for i, value in enumerate(temperatures):
        while waiting and temperatures[waiting[-1]] < value:
            j = waiting.pop()
            out[j] = i - j
        waiting.append(i)
    return out
warmer_days([73, 74, 75, 71, 69, 72, 76, 73]) # -> [1, 1, 4, 2, 1, 1, 0, 0]
```

The top is the most recent unresolved index. Equal temperatures do not resolve a day. Each index is pushed and popped once.

For Largest Rectangle in Histogram, a shorter bar closes the rectangles that taller bars could extend. Store each height with the earliest start where it is valid. A final zero flushes the stack.

```python
def largest_rectangle(heights):
    stack, best = [], 0
    for i, height in enumerate(heights + [0]):
        start = i
        while stack and stack[-1][1] > height:
            start, old_height = stack.pop()
            best = max(best, old_height * (i - start))
        stack.append((start, height))
    return best
largest_rectangle([2, 1, 5, 6, 2, 3]) # -> 10
largest_rectangle([2, 2])             # -> 4
```

For Car Fleet, sort cars from nearest to farthest from the target. Compute each solo arrival time. If it is no greater than the fleet time ahead, it joins that fleet. Otherwise it starts a new fleet. You need only the last fleet time and a count.

Cost: Monotonic scans take $O(n)$ time and $O(n)$ space. Car Fleet also needs sorting..

For a new problem. Name the event that finishes a pending item. Then choose strict or non-strict comparison from the requested relation. A stack is useful only when resolving the most recent item also lets you resolve older ones in order.

8 Binary search: find a boundary

The goal is to discard many candidates after one test. This is safe when a successful midpoint also makes every larger candidate successful. That condition is monotonicity: false values followed by true values. Keep a half-open search range `[lo, hi)`. Return its first true index, or the original `hi` if none exists.

```python
def first_true(lo, hi, test):
    while lo < hi:
        mid = (lo + hi) // 2
        if test(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

a = [1, 2, 2, 4]
first_true(0, len(a), lambda i: a[i] >= 2) # -> 1
first_true(0, len(a), lambda i: a[i] >= 9) # -> 4

def eating_speed(piles, hours):
    def fits(speed):
        return sum((p + speed - 1) // speed for p in piles) <= hours
    return first_true(1, max(piles) + 1, fits)
eating_speed([3, 6, 7, 11], 8)        # -> 4
```

For exact search, find the first value at least the target, then check equality and bounds. For Search a 2D Matrix, use a virtual sorted list: flat index `i` maps to `divmod(i, cols)`. For TimeMap, append timestamps in their guaranteed increasing order. Use `bisect_right(times, t) - 1` for the latest eligible value.

Rotated arrays. With distinct values, `nums[i] <= nums[-1]` is false on the high run and true on the low run. Its first true index is the minimum. For rotated exact search, find this pivot, choose the sorted side that can contain the target, then run ordinary boundary search there. For `[4, 5, 1, 2, 3]`, the pivot is 2.

8.1 A partition instead of a value

Median of Two Sorted Arrays explicitly requires logarithmic time. Choose how many values the shorter array contributes to the left half. The other count is then fixed. Only the two values on each side of each cut matter.

```
def median_sorted(a, b):
    if len(a) > len(b):
        a, b = b, a
    half = (len(a) + len(b) + 1) // 2
    lo, hi = 0, len(a)
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        al = a[i - 1] if i else float("-inf")
        ar = a[i] if i < len(a) else float("inf")
        bl = b[j - 1] if j else float("-inf")
        br = b[j] if j < len(b) else float("inf")
        if al > br:
            hi = i - 1              # too many from a
        elif bl > ar:
            lo = i + 1              # too few from a
        elif (len(a) + len(b)) % 2:
            return max(al, bl)
        else:
            return (max(al, bl) + min(ar, br)) / 2
median_sorted([1, 3], [2])           # -> 2
median_sorted([], [1, 2])            # -> 1.5
```

The left half contains the extra value when the total is odd. At a valid cut, both left boundary values are no greater than both right boundary values.

Cost: Boundary search uses $O(\log R)$ tests. Include the cost of each test. Koko costs $O(n \log M)$..

For a new problem. Write the yes-or-no test and prove why it changes only once. The search variable can be an index, a speed, a capacity, or a partition size. Sorting alone does not prove an arbitrary predicate is monotonic.

9 Linked lists: preserve links before changing them

A pointer is a reference to a node. Changing `cur.next` changes the structure; moving `cur` changes only your local reference. Save the forward link before reversing it. A dummy node gives the head an ordinary predecessor.

```
class Node:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def reverse(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
    return prev
head = reverse(Node(1, Node(2, Node(3))))
head.val, head.next.val, head.next.next.val # -> (3, 2, 1)
```

For Merge Two Sorted Lists, keep a dummy and a tail. Link the smaller head after the tail, advance that input, and move the tail. When one input ends, attach the other remainder. Return `dummy.next`.

For Remove Nth From End, set `slow=dummy` and `fast=head`. Advance `fast` by `n` nodes, then move

both until `fast is None`. Now `slow.next` is the target. Set `slow.next = slow.next.next`. The gap converts a distance from the end into a position from the front.

For Reorder List, the simplest direct representation is an array of node references. Relink them in the order first, last, second, second-last. Set the final `next` to `None`. If constant extra space is required, find the middle, split the list, reverse the second half, and alternate the two halves.

9.1 Reverse a complete group

First prove that `k` nodes exist. Leave an incomplete tail alone. Start `prev` at the node after the group, so reversal already connects the new tail to the remainder.

```
class Node:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
def reverse_groups(head, k):
    dummy = before = Node(0, head)
    while True:
        last = before
        for _ in range(k):
            last = last.next
            if last is None:
                return dummy.next
        after = last.next
        old_first = before.next
        prev, cur = after, old_first
        while cur is not after:
            nxt = cur.next
            cur.next = prev
            prev, cur = cur, nxt
        before.next = last
        before = old_first
head = reverse_groups(Node(1, Node(2, Node(3))), 2)
head.val, head.next.val, head.next.next.val # -> (2, 1, 3)
```

9.2 Cycles and copies

Floyd's cycle test uses a slow pointer and a pointer that moves twice as fast. If a cycle exists, their relative distance changes by one step each round, so they meet. For a linked list, stop safely when `fast` or `fast.next` is absent.

Find the Duplicate Number requires an unchanged array and constant extra space. Read `i -> nums[i]` as a next-pointer function. Index 0 cannot be a target, and values lie in `1..n`. The cycle entrance has two incoming paths, so its index is the duplicate value.

```
def duplicate(nums):
    slow = fast = 0
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break
    start = 0
    while start != slow:
        start = nums[start]
        slow = nums[slow]
    return start
duplicate([1, 3, 4, 2, 2])          # -> 2
```

If the entrance is μ steps from the start and the cycle length is λ, the slow pointer's step count at the meeting is a multiple of λ. Moving one pointer from the start and one from the meeting makes them meet at the entrance after μ steps.

For Copy List With Random Pointer, make `copies = {None: None}`. Pass 1 creates every copied node. Pass 2 sets each copy's `next` and `random` by map lookup. Copy node identities, not values: equal values can belong to different nodes.

9.3 LRU: lookup plus recency order

Use `OrderedDict` for the clearest Python implementation. Every successful read and every write moves the key to the most recent end.

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
cache = LRUCache(2)
cache.put(1, 10)
cache.put(2, 20)
cache.get(1)                        # -> 10
cache.put(3, 30)
cache.get(2)                        # -> -1
```

If asked to build the structure, use a dictionary from keys to doubly linked nodes, with head and tail sentinels. Remove a node with `node.prev.next = node.next` and `node.next.prev = node.prev`. Insert at the recent end by reconnecting both neighboring links. The map supplies the node directly.

Cost: Ordinary list passes take $O(n)$. LRU reads and writes take average $O(1)$..

For a new problem. Use a dummy for changes at the head, a saved successor for rewiring, and a map for cross-links. When a design needs two operations, combine structures that supply those operations.

10 Trees: decide what travels up or down

If an answer needs ancestor information, carry it down. If it needs child answers, combine them on the way up. Use a level frontier when the output groups nodes by distance from the root.

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right
def levels(root):
    frontier, out = [root] if root else [], []
    while frontier:
        out.append([node.val for node in frontier])
        frontier = [child for node in frontier
                    for child in (node.left, node.right) if child]
    return out
root = Node(1, Node(2), Node(3, Node(4)))
levels(root)                          # -> [[1], [2, 3], [4]]
```

The number of levels is maximum depth. The last value of each level is the right side view. For Good Nodes, stack entries are `(node, largest_ancestor)`; count a node when its value is at least that maximum. For Invert Tree, swap the children at every visited node.

10.1 Combine children without deep recursion

Recursion expresses the idea cleanly, but these problems allow long chains. Collect nodes with parents before children, then reverse that order. Both child results exist before the parent is processed.

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right
def tree_summary(root):
    if root is None:
        return 0, True, 0
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        stack.extend(c for c in (node.left, node.right) if c)
    height = {None: 0}
    balanced, diameter = True, 0
    for node in reversed(order):
        left, right = height[node.left], height[node.right]
        height[node] = 1 + max(left, right)
        balanced = balanced and abs(left - right) <= 1
        diameter = max(diameter, left + right)
    return height[root], balanced, diameter
root = Node(1, Node(2, Node(4)), Node(3))
tree_summary(root)                    # -> (3, True, 3)
```

Height counts nodes; diameter counts edges. The path through a node has `left_height + right_height` edges. Compute only the summary needed by the actual problem.

For Maximum Path Sum, use the same order with `gain[None] = 0`. Clamp each child gain to zero. A candidate path is `node.val + left_gain + right_gain`. The gain returned upward is `node.val + max(left_gain, right_gain)`: a path sent to a parent cannot branch. Initialize the global best to

negative infinity so that an all-negative tree still chooses one node. For `[-10, 9, 20, None, None, 15, 7]`, the best path is $15 + 20 + 7 = 42$.

10.2 Use shape and order precisely

Task	Information to keep
Same Tree	Visit node pairs. Two absent nodes match. One absent node or unequal values fails. Compare both child pairs.
Subtree	Try each node as a root and apply Same Tree. Use explicit stacks for deep inputs. This direct $O(nm)$ method fits these sizes.
Validate BST	Stack `(node, low, high)`. Require `low < node.val < high`; narrow the bounds for each child. Checking only the parent is insufficient.
BST ancestor	Move left if both targets are smaller, right if both are larger. The first split is the lowest common ancestor; it can be a target itself.
Kth smallest	Inorder gives sorted values. Push the left spine, pop one node, then enter its right subtree. Stop at the kth pop.
Build from traversals	Preorder supplies the next root. Its unique inorder position splits the subtree. Use an inorder index map and jobs `(parent, side, lo, hi)` on half-open inorder ranges. Skip empty ranges. Otherwise create the root, then schedule right before left so the next job consumes the left preorder root.
Serialize	BFS with explicit null markers. To decode, create the root, then let each queued real node consume the next two child tokens. Values alone do not preserve shape.

For example, serializing a root 1 with only a right child 2 gives `"1,N,2,N,N"` in this BFS format. For repeated kth-smallest queries with updates, maintain subtree sizes: the left size tells you which side contains rank `k`.

Cost: Most traversals take $O(n)$ time. The explicit bottom-up summary uses $O(n)$ space..

For a new problem. Define the helper's return value before its code. Ask whether the parent needs one branch or both. For ordered trees, propagate the full ancestor restriction. For a new tree aggregate, change the per-node combination.

11 Tries: share work across prefixes

A trie stores one character per edge. Words with a common prefix share that path. A terminal marker distinguishes a whole word from a prefix. A dictionary of dictionaries is enough here.

```
def insert(root, word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node["$"] = word                # safe: input uses lowercase letters
def walk(root, text):
    node = root
    for ch in text:
        if ch not in node:
            return None
        node = node[ch]
    return node
root = {}
insert(root, "apple")
walk(root, "app") is not None         # -> True
"$" in walk(root, "app")              # -> False
"$" in walk(root, "apple")            # -> True
```

`setdefault(key, value)` inserts `value` only if the key is absent, then returns the stored value. Search checks the terminal marker. `startsWith` only checks whether the walk succeeds.

For WordDictionary, a literal follows one edge. A dot tries each character child at that position. The pattern must end at a terminal node.

```
def matches(root, pattern):
    def search(node, i):
        if i == len(pattern):
            return "$" in node
        ch = pattern[i]
        if ch == ".":
            return any(search(child, i + 1)
                       for key, child in node.items() if key != "$")
        return ch in node and search(node[ch], i + 1)
    return search(root, 0)
root = {"b": {"a": {"d": {"$": "bad"}}}}
matches(root, ".ad"), matches(root, "b.") # -> (True, False)
```

For Word Search II, put all words in one trie. Combine a grid path with a trie node. If the cell's letter is not a child, stop: no stored word can use that prefix. Mark the cell for the current path, explore neighbors, then restore it. On a terminal marker, save the word and remove that marker so it is reported once. After returning from a child, delete its edge if the child dictionary is empty. This removes exhausted prefixes without reference counters.

Cost: Insert and literal lookup take $O(L)$ per word. Wildcards and grid paths can branch; prefix pruning avoids irrelevant branches..

For a new problem. Use a trie when many queries or candidate words share prefixes. Combine it with a second search only when both searches must advance through the same characters.

12 Heaps: keep the next useful candidate

A heap answers "which candidate is best next?" without sorting after each update. For the kth largest value, keep the largest `k` seen so far in a min-heap. Its smallest value is the weakest retained candidate.

```python
from heapq import heappush, heappop
def kth_largest(nums, k):
    heap = []
    for x in nums:
        heappush(heap, x)
        if len(heap) > k:
            heappop(heap)
    return heap[0]
kth_largest([3, 2, 1, 5, 6, 4], 2)    # -> 5
```

For a stream, preserve the heap between `add` calls. For K Closest Points, sorting by `x*x + y*y` and slicing is the simplest default. For Last Stone Weight, pop the two largest weights from a heap of negatives and push their difference if nonzero.

12.1 Two heaps expose the middle

Keep the lower half in a max-heap and the upper half in a min-heap. All lower values must be no greater than all upper values. Keep equal sizes, or one extra value in the lower half.

```python
from heapq import heappush, heappop
class MedianFinder:
    def __init__(self):
        self.low, self.high = [], []
    def addNum(self, x):
        heappush(self.low, -x)
        heappush(self.high, -heappop(self.low))
        if len(self.high) > len(self.low):
            heappush(self.low, -heappop(self.high))
    def findMedian(self):
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2
finder = MedianFinder()
finder.addNum(1)
finder.addNum(2)
finder.findMedian()                    # -> 1.5
finder.addNum(3)
finder.findMedian()                    # -> 2
```

12.2 Merge streams and enforce cooldowns

For Merge K Sorted Lists, put one front node from each nonempty list in a heap as `(value, list_id, node)`. Pop the smallest, append it, then push its successor from the same list. Only that list's next candidate changes.

Twitter is the same merge in reverse time. Append `(timestamp, tweet_id)` to each author's list. Store followees in a set. A feed merges the user's own list and each followed list. Heap entries are `(-timestamp, user_id, index)`. Pop at most ten tweets; after each pop, push that author's next older index. Follow uses `add`; unfollow uses `discard`. Unique timestamps resolve ties.

For Task Scheduler, fill a round of `cooldown + 1` slots with distinct tasks of largest remaining frequency. Hold their updated counts outside the heap until the round ends.

```
from collections import Counter
from heapq import heapify, heappop, heappush
def schedule_length(tasks, cooldown):
    heap = [-count for count in Counter(tasks).values()]
    heapify(heap)
    elapsed = 0
    while heap:
        pending, used = [], 0
        for _ in range(cooldown + 1):
            if not heap:
                break
            remaining = heappop(heap) + 1
            used += 1
            if remaining:
                pending.append(remaining)
        for remaining in pending:
            heappush(heap, remaining)
        elapsed += cooldown + 1 if heap else used
    return elapsed
schedule_length(list("AAABBB"), 2)    # -> 8
```

The remaining heap after reinsertion means another round is needed, so this round includes idle slots. For `AAABBB` and cooldown 2, the rounds are `AB_ | AB_ | AB`, giving 8 slots. Fill the highest remaining frequencies first so repeated work is spread across rounds. The final round needs no trailing idle time.

Cost: A size-k heap costs $O(n \log(k+1))$*. A merge of k streams costs* $O(N \log(k+1))$*. Median insertion is* $O(\log n)$*..*

For a new problem. Choose the heap key from the next decision, not from the input format. Keep only one front per sorted stream. If old candidates can expire, remove invalid heap tops before using them.

13 Backtracking: choose, explore, undo

Use backtracking when you need to enumerate choices or find a valid arrangement. The state must describe the partial answer and what choices remain. Use a start index when order does not matter. Use unused items when order does matter.

```
def combinations(candidates, target):
    candidates = sorted(candidates)
    out, path = [], []
    def search(start, remaining):
        if remaining == 0:
            out.append(path.copy())
            return
        for i in range(start, len(candidates)):
            x = candidates[i]
            if x > remaining:
                break
            path.append(x)
            search(i, remaining - x)  # same index allows reuse
            path.pop()
    search(0, target)
    return out
combinations([2, 3, 6, 7], 7)        # -> [[2, 2, 3], [7]]
```

For Combination Sum II, recurse with `i + 1`, and skip when `i > start and a[i] == a[i-1]`. This skips duplicate choices at the same depth, while allowing equal values at different depths. The early break requires sorted positive values.

Choice structure	Minimal change
Subsets	Start with `[[]]`. For each value, extend with `[part + [x] for part in out]`. The right side is built before extension.
Subsets with duplicates	Sort. Save the current path at each call; loop from `start`, skip equal same-depth choices, recurse at `i+1`.
Permutations	Pick each remaining item in turn; recurse on the remaining pool. Save the path when the pool is empty.
Phone letters	Start with `[""]`. For each digit, form all old prefixes plus each mapped letter.
Generate parentheses	Track opens and closes. Add an opener if `opens < n`; add a closer if `closes < opens`. Save at length `2*n`.
Palindrome partitions	Try every next piece `s[start:end]`. Continue only if it equals its reverse. Save when `start == len(s)`.
N-Queens	Place one queen per row. Track used columns and diagonal keys `r-c`, `r+c`. Mark before recursion; remove after it.

Examples: `[1,2]` gives subsets `[[],[1],[2],[1,2]]`. Digits `"23"` give the nine pairs from `"abc"` and `"def"`. For a 4-by-4 queens board, columns `[1,3,0,2]` form one valid arrangement.

13.1 A grid path owns its visited cells

Word Search cannot reuse a cell in one word. Mark cells for the current path, then undo the mark even when a child succeeds.

```
def word_exists(board, word):
    rows, cols = len(board), len(board[0])
    used = set()
    def search(r, c, i):
        if i == len(word):
            return True
        if not (0 <= r < rows and 0 <= c < cols):
            return False
        if (r, c) in used or board[r][c] != word[i]:
            return False
        used.add((r, c))
        found = any(search(r + dr, c + dc, i + 1)
                    for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)])
        used.remove((r, c))
        return found
    return any(search(r, c, 0) for r in range(rows) for c in range(cols))
word_exists([list("AB"), list("CD")], "ABD") # -> True
word_exists([list("AB")], "ABA")      # -> False
```

Cost: Search can be exponential. Estimate branch count, depth, and the unavoidable output size..

For a new problem. Decide whether choices are distinct by position, value, or order. Prune only when no completion can work. If many branches reach the same future state and only a count or optimum is needed, test DP instead.

14 Graph traversal: choose the state and the frontier

A graph can be an adjacency list, a grid, or a set of legal transformations. DFS explores with a stack. BFS explores with a queue and gives shortest paths when every move costs the same. Mark a state when adding it to the frontier so that it is added once.

14.1 A component is one complete flood

Scan all possible starts. Each unvisited start begins a new connected component. This version sinks land as it visits it, so it changes the grid.

```
def island_sizes(grid):
    rows, cols = len(grid), len(grid[0])
    sizes = []
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 1:
                continue
            grid[r][c] = 0
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                    a, b = x + dx, y + dy
                    if 0 <= a < rows and 0 <= b < cols and grid[a][b] == 1:
                        grid[a][b] = 0
                        stack.append((a, b))
            sizes.append(size)
    return sizes
island_sizes([[1, 1, 0], [0, 0, 1]])  # -> [2, 1]
```

Number of Islands uses string cells `"1"` and `"0"`; adjust the two literals. Max Area of Island uses integer cells; return the largest size, or 0 if none. For graph components, build `adj = [[] for _ in range(n)]`, add both directions, and use a visited array. Include isolated nodes in the outer scan.

Reverse the question when there are many starts. For Pacific Atlantic, flood inland from each ocean border, moving to equal or higher neighbors. Intersect the two reached sets. The reverse move represents a forward downhill path. For Surrounded Regions, flood border-connected `"O"` cells as safe, then flip only the remaining `"O"` cells.

14.2 Many sources, one BFS

To find distance to the nearest gate, seed all gates at distance zero. Every unfilled neighbor receives the current distance plus one.

```
from collections import deque
def fill_distances(rooms):
    rows, cols, INF = len(rooms), len(rooms[0]), 2**31 - 1
    q = deque((r, c) for r in range(rows) for c in range(cols)
              if rooms[r][c] == 0)
    while q:
        r, c = q.popleft()
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            a, b = r + dr, c + dc
            if 0 <= a < rows and 0 <= b < cols and rooms[a][b] == INF:
                rooms[a][b] = rooms[r][c] + 1
                q.append((a, b))
rooms = [[0, -1], [2**31 - 1, 2**31 - 1]]
fill_distances(rooms)
rooms                                # -> [[0, -1], [1, 2]]
```

For Rotting Oranges, start with all rotten cells and count fresh cells. Process `len(q)` items per round; only then increase minutes. Stop when fresh is zero or the queue is empty. Return -1 if fresh cells remain; return 0 if none were fresh initially.

Word Ladder is BFS on words. A neighbor changes one character and belongs to the unvisited word set. Remove words when enqueuing. Start the sequence length at 1 because the output counts words, not edges. Alternatively, group words by a one-character wildcard key and clear each bucket after using it.

Clone Graph. A map from original node to copied node is also the visited structure. Create and record a copy before processing its neighbors, so a cycle finds the existing copy. Append the neighbor copies in the original edge order. An absent start returns `None`.

Cost: An adjacency-list traversal takes $O(V + E)$ time. A four-neighbor grid traversal takes $O(RC)$..

For a new problem. List the facts that define a state. If arriving at the same location with a different key, budget, or move count changes the future, that fact belongs in the state. Global visited marks mean "this state is finished"; backtracking marks mean "this path owns it for now".

15 Graph structure: readiness and connectivity

15.1 Topological order: take what is ready

An edge `u -> v` means `u` must happen first. Indegree counts unmet prerequisites. Removing a ready node reduces its dependents' counts.

```
from collections import deque
def topological_order(n, edges):
    adj = [[] for _ in range(n)]
    indegree = [0] * n
    for u, v in edges:
        adj[u].append(v)
        indegree[v] += 1
    q = deque(i for i in range(n) if indegree[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                q.append(v)
    return order if len(order) == n else []
topological_order(3, [(0, 1), (1, 2)]) # -> [0, 1, 2]
topological_order(2, [(0, 1), (1, 0)]) # -> []
```

For Course Schedule input `[course, prerequisite]`, add `prerequisite -> course`. Course Schedule I asks whether all nodes are removed; II asks for the order. A cycle leaves nodes with unmet prerequisites.

Alien Dictionary first builds the graph. Include every observed character, even if it has no edge. Compare adjacent words. Only the first differing characters establish an edge. Reject a longer word before its own prefix, such as `"abc"` before `"ab"`. Use adjacency sets so duplicate evidence does not increase indegree twice. Then apply Kahn's algorithm with dictionary keys.

15.2 Union-find: are the endpoints already connected?

Use union-find when edges arrive and component membership is the needed information. Path compression shortens future root searches. Attach the smaller tree to the larger one.

```
class DSU:
    def __init__(self, n):
        self.parent, self.size = list(range(n)), [1] * n
    def find(self, x):
        while x != self.parent[x]:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        return True
dsu = DSU(3)
dsu.union(0, 1), dsu.union(1, 2), dsu.union(0, 2) # -> (True, True, False)
```

For Redundant Connection, scan edges in input order; a failed union identifies the extra cycle edge. For Connected Components, start at `n` and subtract one per successful union, or use a simpler flood fill. For

Graph Valid Tree, require exactly `n - 1` edges and connectivity. A single flood from node 0 checks the latter.

Cost: Topological sort takes $O(V+E)$. Union-find operations take amortized $O(\alpha(V))$, effectively constant at these sizes..

For a new problem. Use directed dependencies for ordering, undirected components for connectivity. A visited directed node does not by itself prove a cycle; DFS needs a separate active-path state, or use Kahn's readiness count.

16 Weighted paths and using every edge

16.1 Dijkstra: finish the cheapest reachable state

BFS cannot finalize unequal-cost paths by queue order. With nonnegative edges, the smallest queued distance cannot improve after it is popped with its current best value.

```
from heapq import heappop, heappush
def dijkstra(adj, start):
    distance = [float("inf")] * len(adj)
    distance[start] = 0
    heap = [(0, start)]
    while heap:
        cost, u = heappop(heap)
        if cost != distance[u]:
            continue                # obsolete entry
        for v, weight in adj[u]:
            candidate = cost + weight
            if candidate < distance[v]:
                distance[v] = candidate
                heappush(heap, (candidate, v))
    return distance
dijkstra([[(1, 5), (2, 1)], [], [(1, 1)]], 0) # -> [0, 2, 1]
```

Network Delay returns the largest shortest distance, or -1 if any node is unreachable. Swim in Rising Water changes the path cost: extend with `max(cost, neighbor_height)` instead of addition. Start at the first cell's height. Pop the destination to obtain the minimum required water level. The key property remains: extending a path cannot reduce its cost.

16.2 A move budget needs a layer

Cheapest Flights Within K Stops allows at most `k + 1` edges. Use one relaxation round per allowed edge. Read old distances and write to a copy.

```
def cheapest(n, flights, start, end, k):
    distance = [float("inf")] * n
    distance[start] = 0
    for _ in range(k + 1):
        new = distance.copy()
        for u, v, price in flights:
            new[v] = min(new[v], distance[u] + price)
        distance = new
    return -1 if distance[end] == float("inf") else distance[end]
cheapest(3, [(0, 1, 1), (1, 2, 1), (0, 2, 9)], 0, 2, 0) # -> 9
```

In-place writes would allow a later edge in the same round to use an extra flight.

16.3 Connect all points: cheapest edge into a growing tree

Prim's algorithm keeps each outside point's cheapest connection to the current tree. Choose the cheapest one, then update the others. For this dense complete graph, a simple scan avoids a huge edge list.

```
def connect_points(points):
    n = len(points)
    best, used = [float("inf")] * n, [False] * n
    best[0], total = 0, 0
    for _ in range(n):
        u = min((i for i in range(n) if not used[i]), key=lambda i: best[i])
        total += best[u]
        used[u] = True
        x, y = points[u]
        for v, (a, b) in enumerate(points):
            if not used[v]:
                best[v] = min(best[v], abs(x - a) + abs(y - b))
    return total
connect_points([[0, 0], [2, 2], [3, 0]]) # -> 6
```

The cheapest edge crossing from the tree to the outside can belong to a minimum spanning tree. Prim stores edge cost, while Dijkstra stores full path cost. They answer different questions.

16.4 Itinerary: finish dead ends last

Reconstruct Itinerary must consume every ticket exactly once, including duplicate tickets. Use an Euler trail. Take smallest destinations first, but append airports only when no outgoing ticket remains. Reverse that completion order.

```
from collections import defaultdict
def itinerary(tickets):
    adj = defaultdict(list)
    for src, dst in sorted(tickets, reverse=True):
        adj[src].append(dst)
    stack, finished = ["JFK"], []
    while stack:
        if adj[stack[-1]]:
            stack.append(adj[stack[-1]].pop())
        else:
            finished.append(stack.pop())
    return finished[::-1]
itinerary([("JFK", "KUL"), ("JFK", "NRT"), ("NRT", "JFK")])
# -> ['JFK', 'NRT', 'JFK', 'KUL']
```

The statement guarantees a valid itinerary. `KUL` is selected first but has no exit, so it belongs at the end. This explains why simply recording greedy forward choices fails.

Cost: Dijkstra is $O((V+E)\log V)$ on these simple graphs; limited relaxation is $O((k+1)E)$. Dense Prim is $O(n^2)$ time and $O(n)$ space. Itinerary sorting is $O(E \log E)$..

For a new problem. Define what a path costs and whether extending it can lower that cost. If remaining moves change the future, add a layer. If every edge must be used, reason about edges rather than marking each vertex once.

17 DP: keep only the facts that change the future

DP merges repeated subproblems. Define a state in one sentence, list its legal choices, and combine their answers. Use `min` for minimum cost, `max` for maximum value, `or` for existence, and addition for counts of distinct choices. Choose the base case from that meaning.

17.1 Take or skip; extend or restart

For House Robber, the best result for a prefix either skips its last house or takes it and skips its neighbor.

```
def rob(nums):
    best = [0, 0]
    for money in nums:
        best.append(max(best[-1], best[-2] + money))
    return best[-1]
rob([2, 7, 9, 3, 1])                # -> 12
```

For House Robber II, handle one house separately. Otherwise return the larger of `rob(nums[1:])` and `rob(nums[:-1])`. A valid circular plan must exclude at least one endpoint.

For Maximum Subarray, define `ending` as the best nonempty subarray ending here. Extend the previous subarray or start a new one. Keep a separate best over all endpoints.

```
def max_subarray(nums):
    ending = best = nums[0]
    for x in nums[1:]:
        ending = max(x, ending + x)
        best = max(best, ending)
    return best
max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]) # -> 6
max_subarray([-3, -1])                # -> -1
```

For Maximum Product Subarray, retain both extremes. A negative value can turn the smallest old product into the largest new one.

```
def max_product(nums):
    high = low = best = nums[0]
    for x in nums[1:]:
        high, low = max(x, x * high, x * low), min(x, x * high, x * low)
        best = max(best, high)
    return best
max_product([-2, 3, -4])              # -> 24
max_product([-2, 0, -1])              # -> 0
```

Simultaneous assignment makes both new extremes read the old extremes.

For the stock row, `free[i]` is the best future profit from day `i` with no share; `hold[i]` is the best future profit with a share already held. Buying pays the price; selling receives it. Selling also skips the next day.

State	Transition and base
Stair ways `ways[i]`	Last move came from `i-1` or `i-2`. Add both counts. `ways[0]=ways[1]=1`.
Stair cost `cost_to[i]`	Cost to reach position `i`: take the smaller of `cost_to[i-1]+cost[i-1]` and `cost_to[i-2]+cost[i-2]`. Start positions 0 and 1 cost zero; answer is position `n`.

State	Transition and base
Decode Ways `ways[i]`	Count decodings of `s[:i]`. Add `ways[i-1]` if the last digit is not 0. Add `ways[i-2]` if the last pair is 10 through 26. Start `ways[0]=1`.
Word Break `possible[i]`	Can `s[:i]` be split? Try a last dictionary word: `possible[j]` and `s[j:i] in words`. Start `possible[0]=True`; limit piece lengths to the longest word.
Stock cooldown	`free[i]=max(free[i+1], -price[i]+hold[i+1])`; `hold[i]=max(hold[i+1], price[i]+free[i+2])`. Fill backward with zero entries after the last day. Selling skips one day.

For Decode Ways, `"06"` gives 0 and `"226"` gives 3. For Word Break, `"applepenapple"` splits with `"apple"` and `"pen"`. For cooldown prices `[1,2,3,0,2]`, the best profit is 3.

17.2 Subsequence: an endpoint state

A subsequence preserves order but can skip positions. For LIS, define `length[i]` as its best length ending at index `i`. Try each earlier smaller value as the previous endpoint.

```
def lis(nums):
    length = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                length[i] = max(length[i], length[j] + 1)
    return max(length, default=0)
lis([10, 9, 2, 5, 3, 7, 101, 18])   # -> 4
```

This $O(n^2)$ form fits the base limit of 2500. The follow-up asks for $O(n \log n)$. For that, keep the smallest possible tail for each length. A smaller tail gives every future value at least as many ways to extend.

```
from bisect import bisect_left
def lis_fast(nums):
    tails = []
    for x in nums:
        i = bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)
lis_fast([2, 2, 2])                  # -> 1
lis_fast([3, 5, 6, 2, 4])            # -> 3
```

The tails list gives the length; its final contents need not be an actual subsequence.

For a new problem. Ask whether two histories with the same proposed state always have the same available futures. If not, add the missing fact. If yes, keep only the best result or the count for that state. Use a full table first; compress it only after its dependencies are clear.

18 DP on totals: distinguish reuse and order

Coin Change asks for minimum count. For each total, try its last coin.

```
def fewest_coins(coins, amount):
    dp = [0] + [float("inf")] * amount
    for total in range(1, amount + 1):
        for coin in coins:
            if coin <= total:
                dp[total] = min(dp[total], 1 + dp[total - coin])
    return -1 if dp[amount] == float("inf") else dp[amount]
fewest_coins([1, 3, 4], 6)          # -> 2
fewest_coins([2], 3)                # -> -1
```

Greedily taking the largest coin gives `4+1+1` on the first example; DP finds `3+3`. For counts, the empty total has one construction. Loop order determines which constructions are distinct.

```
def coin_combinations(coins, amount):  # Coin Change II: order ignored
    ways = [1] + [0] * amount
    for coin in coins:
        for total in range(coin, amount + 1):
            ways[total] += ways[total - coin]
    return ways[amount]
def ordered_sums(nums, target):       # Combination Sum IV: order matters
    ways = [1] + [0] * target
    for total in range(1, target + 1):
        for x in nums:
            if x <= total:
                ways[total] += ways[total - x]
    return ways[target]
coin_combinations([1, 2, 3], 4)      # -> 4
ordered_sums([1, 2, 3], 4)           # -> 7
```

Coin-first counts each multiset once: when processing coin `c`, all counted choices use only coin types processed so far. Total-first distinguishes the last element, so `1+3` and `3+1` are different sequences. These recurrences require positive denominations. With zero or negative values and unrestricted reuse, counts can be infinite.

18.1 Use each item once

For Equal Subset Partition, an even total is necessary. Ask whether half the total is reachable. Build new sums from an unchanged old set so that each item is used once.

```
def equal_partition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target, reachable = total // 2, {0}
    for x in nums:
        reachable |= {s + x for s in reachable if s + x <= target}
    return target in reachable
equal_partition([1, 5, 11, 5])       # -> True
equal_partition([1, 2, 5])           # -> False
```

The right-hand set is built completely before the union changes `reachable`. For a 1-D Boolean table, scan totals downward to prevent reuse of the current item. For unlimited reuse, scan upward so the current item can extend a total it already helped form.

Target Sum makes a sign choice for each index. Two paths with the same index and sum have the same remaining choices.

```python
from functools import cache
def target_ways(nums, target):
    @cache
    def ways(i, total):
        if i == len(nums):
            return int(total == target)
        return ways(i + 1, total + nums[i]) + ways(i + 1, total - nums[i])
    return ways(0, 0)
target_ways([1, 1, 1, 1, 1], 3)      # -> 5
target_ways([0, 0], 0)               # -> 4
```

Zeros still have two sign choices. Do not deduplicate paths when the output counts assignments.

Cost: Total-based tables usually take $O(nT)$ time, where T is the tracked total range..

For a new problem. Before writing loops, answer three questions: can an item repeat, does order matter, and do you need existence, a count, or an optimum? Those answers determine the state, update order, and combining operation.

19 DP on two sequences, grids, and intervals

Two indexes often mean "how much of each input has been consumed?". Use a prefix convention consistently. The extra row and column describe empty prefixes.

```python
def lcs(a, b):
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[-1][-1]
lcs("abcde", "ace")                  # -> 3
```

Problem and state	Legal transition
Edit Distance: transform `a[:i]` into `b[:j]`	If final characters match, take the diagonal. Otherwise take 1 plus the minimum of up (delete), left (insert), and diagonal (replace). Initialize `dp[i][0]=i`, `dp[0][j]=j`.
Distinct Subsequences: form `t[:j]` from `s[:i]`	Always add `dp[i-1][j]` to skip the last source character. If it matches, also add `dp[i-1][j-1]` to use it. Every `dp[i][0]=1`; `dp[0][j]=0` for positive `j`.
Interleaving: form `s3[:i+j]` from both prefixes	Reject unequal total length first. Take from the first string if its last character matches and `dp[i-1][j]` is true; similarly from the second via `dp[i][j-1]`. Start `dp[0][0]=True`.
Unique Paths: paths to cell `(r,c)`	Add from above and left. First row and first column are 1. Answer is the bottom-right cell.
Increasing grid path: best path starting at `(r,c)`	Sort cells by height, descending. Store 1 plus the largest answer among strictly higher neighbors, or 1 if none. Larger neighbors are already finished. Return the largest stored answer.

Examples: `"horse" -> "ros"` needs 3 edits. There are 3 ways to select `"rabbit"` from `"rabbbit"`. `"ab"`

and `"cd"` can interleave as `"acbd"`. A 3-by-2 right/down grid has 3 paths. Grid `[[1,2],[4,3]]` has a strictly increasing path of length 4.

19.1 Palindromes: expand from their centers

An odd palindrome has one center; an even palindrome has a gap center. Expand while opposite characters match. Each successful expansion is one palindrome.

```
def palindrome_summary(s):
    count, start, size = 0, 0, 0
    for i in range(len(s)):
        for left, right in [(i, i), (i, i + 1)]:
            while left >= 0 and right < len(s) and s[left] == s[right]:
                count += 1
                if right - left + 1 > size:
                    start, size = left, right - left + 1
                left -= 1
                right += 1
    return count, s[start:start + size]
palindrome_summary("aaa")            # -> (6, 'aaa')
palindrome_summary("cbbd")           # -> (5, 'bb')
```

Use the count for Palindromic Substrings, or the longest range for Longest Palindromic Substring. Save ranges during the scan; avoid copying a growing string at each expansion.

19.2 Regex: branch on a pattern unit

Here a dot matches one character; a star repeats its preceding unit zero or more times. This is a whole-string match, not substring search.

```
from functools import cache
def regex_match(s, pattern):
    @cache
    def match(i, j):
        if j == len(pattern):
            return i == len(s)
        first = i < len(s) and pattern[j] in (s[i], ".")
        if j + 1 < len(pattern) and pattern[j + 1] == "*":
            return match(i, j + 2) or (first and match(i + 1, j))
        return first and match(i + 1, j + 1)
    return match(0, 0)
regex_match("aab", "c*a*b")          # -> True
regex_match("ab", ".*c")            # -> False
```

At a star, skip the whole two-character unit, or consume one matching input character and keep the same unit. These are exhaustive choices. The cache bounds the work by pairs of positions.

19.3 Burst Balloons: choose the last action

Choosing the first balloon changes the neighbors of later choices. Choosing the last balloon in an interval fixes its two neighbors: the interval's outside walls.

```python
def burst(nums):
    a = [1] + nums + [1]
    dp = [[0] * len(a) for _ in a]
    for width in range(2, len(a)):
        for left in range(len(a) - width):
            right = left + width
            for last in range(left + 1, right):
                score = dp[left][last] + dp[last][right]
                score += a[left] * a[last] * a[right]
                dp[left][right] = max(dp[left][right], score)
    return dp[0][-1]
burst([3, 1, 5, 8])                  # -> 167
```

`dp[left][right]` covers the open interval between two unburst walls. Fill narrower intervals first so both sides of `last` are available.

Cost: Two-sequence tables take $O(mn)$ time and space. Center expansion is $O(n^2)$ time and $O(1)$ extra space. Sorted grid DP is $O(RC \log(RC))$. Burst Balloons is $O(n^3)$ time and $O(n^2)$ space..

For a new problem. Define the boundary of a solved subproblem. If the first action leaves unstable boundaries, try fixing the last action. If two inputs are consumed independently, test two position indexes. Do not copy a recurrence until its state meaning matches.

20 Greedy: prove that alternatives can be discarded

A locally attractive choice is not enough. Show that it preserves a possible optimum or that every discarded choice must fail. Often the proof gives a small running summary.

20.1 Reachability and a failed prefix

Jump Game keeps the farthest reachable index. Every earlier index in that prefix is reachable too.

```python
def can_jump(nums):
    farthest = 0
    for i, reach in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + reach)
    return True
can_jump([2, 3, 1, 1, 4])            # -> True
can_jump([3, 2, 1, 0, 4])            # -> False
```

For Jump Game II, view the reachable indices as BFS layers. Start `left=right=jumps=0`. While `right < n-1`, compute the largest `i+nums[i]` over `left..right`. Set the next layer to `[right+1, farthest]`, then increase jumps. The input guarantees reachability. For `[2,3,1,1,4]`, two layers reach the end.

For Gas Station, total gas must cover total cost. Scan with a candidate start and a running tank. If the tank becomes negative at `i`, restart at `i+1` and reset the tank. Every intermediate prefix from the old candidate was nonnegative; a later start inside that failed segment has no more fuel at `i`. Thus the whole segment can be discarded. For `gas=[1,2,3,4,5]`, `cost=[3,4,5,1,2]`, return 3.

20.2 Keep only the necessary summary

Problem	Summary and reason
Buy/Sell Stock	Treat today as the sell day. Compare its price with the minimum earlier price. Update the minimum and best profit; start best at zero.
Hand of Straights	Count cards and scan sorted values. The smallest unused card must start a group. Consume one copy of each of the next `groupSize` consecutive values; fail if any is missing. Reject a nondivisible total first.
Merge Triplets	Discard triplets exceeding the target in any coordinate. Merge all others by coordinate-wise maximum. They can only help; compare the final triple with the target.
Partition Labels	Store each character's last index. Extend the current part's end to every last index encountered. Cut as soon as the scan reaches that end. Any earlier cut would split a character.

Examples: stock prices `[7,1,5,3,6,4]` give profit 5. Cards `[1,2,3,3,4,5]` form two groups of size 3. `"abac"` partitions as lengths `[3,1]`. Triplets `[2,5,3]` and `[1,7,5]` merge to `[2,7,5]`.

20.3 A range can summarize all feasible states

For Valid Parenthesis String, `low` and `high` are the minimum and maximum feasible numbers of unmatched openers. A star can close, do nothing, or open, so the feasible counts remain a contiguous range.

```
def valid_stars(s):
    low = high = 0
    for ch in s:
        if ch == "(":
            low, high = low + 1, high + 1
        elif ch == ")":
            low, high = low - 1, high - 1
        else:
            low, high = low - 1, high + 1
        if high < 0:
            return False
        low = max(low, 0)
    return low == 0
valid_stars("(*))"), valid_stars("*(") # -> (True, False)
```

If even the maximum is negative, all choices have too many closers. Clamping the minimum removes invalid negative possibilities. At the end, zero must be feasible.

For a new problem. Look for an exchange argument, a forced smallest choice, or a whole region of failed candidates. If you cannot explain what is safely discarded, test a counterexample and keep more state with DP.

21 Intervals: choose a sweep order

Decide what equality means first. Merge Intervals treats touching closed intervals as overlapping. Meetings can reuse a room exactly when another meeting ends.

```
def merge_intervals(intervals):
    out = []
    for start, end in sorted(intervals):
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out
merge_intervals([[1, 4], [4, 5], [8, 9]]) # -> [[1, 5], [8, 9]]
```

For Insert Interval, the input is already sorted and disjoint. Copy intervals entirely before the new interval, absorb overlaps into it, then append intervals entirely after it. For Non-overlapping Intervals, sort by end and keep each interval whose start is at least the last kept end. Choosing the earliest finish leaves at least as much room for future intervals. Return total minus kept. For Meeting Rooms I, sort by start and reject any neighboring overlap.

21.1 Count simultaneous activity

Represent each start by $+1$ and each end by -1. Sort events and track the maximum running count. At equal times, process ends before starts; tuple sorting does this because $-1 < +1$.

```
def meeting_rooms(intervals):
    events = [(start, 1) for start, end in intervals]
    events += [(end, -1) for start, end in intervals]
    active = best = 0
    for time, change in sorted(events):
        active += change
        best = max(best, active)
    return best
meeting_rooms([(0, 30), (5, 10), (15, 20)]) # -> 2
meeting_rooms([(0, 5), (5, 9)])       # -> 1
```

21.2 Sort queries to make candidate changes one-way

For Minimum Interval to Include Each Query, process queries in increasing order. Push intervals that have started, ordered by length. Pop expired intervals whenever they reach the top.

```
from heapq import heappush, heappop
def min_intervals(intervals, queries):
    intervals = sorted(intervals)
    heap, i, answers = [], 0, {}
    for query in sorted(queries):
        while i < len(intervals) and intervals[i][0] <= query:
            start, end = intervals[i]
            heappush(heap, (end - start + 1, end))
            i += 1
        while heap and heap[0][1] < query:
            heappop(heap)
        answers[query] = heap[0][0] if heap else -1
    return [answers[q] for q in queries]
min_intervals([[1, 4], [2, 4], [3, 6], [4, 4]], [2, 3, 4, 5])
# -> [3, 3, 1, 4]
```

Expired entries may remain below the top. That is safe: only a valid top is used for an answer. Intervals

are closed here, so size is `end - start + 1` and expiration is strictly `end < query`.

Cost: Ordinary sweeps take $O(n \log n)$ including sorting. Query sweep takes $O((n+q)\log(n+q))$..

For a new problem. Choose an order in which items only enter, leave, or become final. If answers must retain input order, save them by original index or query value and restore that order afterward.

22 Matrices and number operations

22.1 Separate reading from changing

Set Matrix Zeroes must react only to original zeros. First record affected rows and columns, then write zeros. For example, `[[1,2],[3,0]]` becomes `[[1,0],[0,0]]`. If constant space is required, use the first row and first column as markers. Save whether each originally contained a zero. Mark from the interior, update the interior, then update those two boundaries last.

To rotate a square matrix clockwise in place, transpose it, then reverse each row. The coordinate mapping is `(r,c) -> (c,n-1-r)`.

```
def rotate(matrix):
    for r in range(len(matrix)):
        for c in range(r + 1, len(matrix)):
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
matrix = [[1, 2], [3, 4]]
rotate(matrix)
matrix                              # -> [[3, 1], [4, 2]]
```

For Spiral Matrix, track the unvisited rectangle with `top, bottom, left, right`. Read its top edge and increase top; read its right edge and decrease right. If rows remain, read the bottom edge backward and decrease bottom. If columns remain, read the left edge upward and increase left. Recheck the corresponding bounds before each edge to avoid duplicating a single row or column. `[[1,2,3],[4,5,6]]` gives `[1,2,3,6,5,4]`.

22.2 Digits: process one place and carry

For Plus One, scan right to left. Replace a 9 with 0 and continue; otherwise increment and return. If all digits were 9, prepend 1. Thus `[9,9]` becomes `[1,0,0]`. For Add Two Numbers, digits arrive least significant first. Loop while either list or carry remains. Add available digits and carry; write `total % 10`, carry `total // 10`. Use a dummy and tail to build the result. A final carry requires a new node.

For Multiply Strings, reverse digit lists so index `i` means the 10^i place. A pair product belongs in position `i+j`. Normalize carries afterward.

```
def multiply(a, b):
    x, y = [int(ch) for ch in a[::-1]], [int(ch) for ch in b[::-1]]
    slots = [0] * (len(x) + len(y))
    for i, left in enumerate(x):
        for j, right in enumerate(y):
            slots[i + j] += left * right
    for i in range(len(slots) - 1):
        carry, slots[i] = divmod(slots[i], 10)
        slots[i + 1] += carry
    while len(slots) > 1 and slots[-1] == 0:
        slots.pop()
    return "".join(str(digit) for digit in slots[::-1])
multiply("123", "456"), multiply("0", "99") # -> ('56088', '0')
```

Reverse Integer forbids overflow beyond signed 32 bits. Work with a nonnegative magnitude and a sign-specific limit. Check before multiplying, rather than constructing an out-of-range intermediate value.

```
def reverse_integer(x):
    sign = -1 if x < 0 else 1
    limit = 2**31 if sign < 0 else 2**31 - 1
    x, result = abs(x), 0
    while x:
        x, digit = divmod(x, 10)
        if result > (limit - digit) // 10:
            return 0
        result = result * 10 + digit
    return sign * result
reverse_integer(-120), reverse_integer(1534236469) # -> (-21, 0)
```

22.3 Halve repeated work; detect repeated states

For powers, compute the half-power once, square it, and multiply by the base if the exponent is odd.

```
def power(x, n):
    if n < 0:
        return power(1 / x, -n)
    if n == 0:
        return 1
    half = power(x, n // 2)
    return half * half * (x if n % 2 else 1)
power(2, -3), power(3, 5)            # -> (0.125, 243)
```

For Happy Number, repeatedly replace the value with the sum of its squared digits. Stop at 1 or when the value repeats in a seen set. For 19: $19 \to 82 \to 68 \to 100 \to 1$. For Missing Number, subtract the actual sum from `n * (n + 1) // 2`. Python's integer arithmetic makes this direct formula safe for these bounds.

22.4 Geometry: fix enough points to force the rest

Detect Squares stores a `Counter` of coordinate tuples, including duplicate copies. Fix the query and a stored diagonal corner; the other two corners are then forced.

```
from collections import Counter
def squares(points, query):
    counts = Counter(map(tuple, points))
    x, y = query
    result = 0
    for (a, b), copies in list(counts.items()):
        if a != x and abs(a - x) == abs(b - y):
            result += copies * counts[(a, y)] * counts[(x, b)]
    return result
squares([(3, 10), (11, 2), (3, 2), (11, 2)], (11, 10)) # -> 2
```

Cost: Matrix scans are $O(RC)$; digit multiplication is $O(mn)$; powers use $O(\log |n|)$ recursive levels. Square queries scan distinct stored points..

For a new problem. Use a coordinate mapping for transformations and place values for arithmetic. If each step is deterministic, a repeated state implies a cycle. For geometric counts, enumerate the smallest set of points that fixes the remaining coordinates.

23 Bits: operate on a fixed representation

For nonnegative integers, `n & 1` reads the low bit, `n >> 1` removes it, and `n << 1` shifts left. XOR cancels equal pairs: `x ^ x == 0` and `x ^ 0 == x`.

```
answer = 0
for x in [4, 1, 2, 1, 2]:
    answer ^= x
answer                                  # -> 4
n = 11                                  # binary 1011
n & 1, n >> 1, n.bit_count()            # -> (1, 5, 3)
counts = [0] * 6
for i in range(1, 6):
    counts[i] = counts[i >> 1] + (i & 1)
counts                                  # -> [0, 1, 1, 2, 1, 2]
```

For Number of 1 Bits, `bit_count()` is the direct Python operation. To implement it, add the low bit and shift until zero; alternatively `n &= n-1` removes one set bit per step. For Reverse Bits, do exactly 32 steps, including leading zeros:

```
def reverse_bits(n):
    result = 0
    for _ in range(32):
        result = (result << 1) | (n & 1)
        n >>= 1
    return result
reverse_bits(1)                         # -> 2147483648
```

For Sum of Two Integers, XOR adds without carry; shifted AND supplies the carry. Python negative integers do not have a fixed width, so mask both updates to 32 bits.

```python
def bit_add(a, b):
    mask = (1 << 32) - 1
    a, b = a & mask, b & mask
    while b:
        a, b = (a ^ b) & mask, ((a & b) << 1) & mask
    return a if a < (1 << 31) else ~(a ^ mask)
bit_add(-3, 5), bit_add(-4, -7)       # -> (2, -11)
```

The final expression converts a negative two's-complement bit pattern to a Python negative integer. In these problems, 32-bit loops have constant cost. For arbitrary widths, count the bits processed.

For a new problem. Identify the bitwise property that matches the task: parity, cancellation, a digit recurrence, or carry. Define the width and signed interpretation before using shifts on negative values.

24 Recall and transfer in a few minutes

Pick one problem without reading its technique label. State the needed information, the rule that preserves it, and the loop shape. Write a small working example. Then change one rule and decide whether the same argument survives.

Change to test	What must change?
Two Sum now asks for the number of index pairs.	Store frequencies, add the partner count before recording the current value.
A valid window has at most two distinct symbols.	Store symbol counts; delete a key when its count reaches zero. Shrink while more than two keys remain.
A range-sum task allows negative values.	The usual sum-limit window can fail. For exact sums, use prefix differences and a count map.
Moves have unequal nonnegative costs.	Queue distance is no longer final. Use Dijkstra with a heap.
The same coin can be used only once.	Use an old/new layer or a descending total loop.
An ordered sum becomes an unordered combination.	Process one value type at a time so different orders are counted once.
A tree has 100,000 nodes in one chain.	Preserve the recurrence with a stack or an explicit dependency order.
The answer is a path, not only its cost.	Store the predecessor when a best state improves, then reconstruct from the destination.

Check one smallest case, one case that activates the main loop, and one case that challenges the key assumption. Useful choices are duplicates, all-negative values, a missing answer, a disconnected state, and equal boundaries. Use only cases allowed by the statement. When stuck, return to the exact question your data structure must answer.

25 Problem map: all 151 problems

Use a row to recall the required change, then open its linked technique if you cannot rebuild it. The problem number links to the LeetCode statement. Technique references link inside this PDF. The code examples in earlier sections teach the shared operations; these rows supply each problem's distinguishing rule.

25.1 Arrays & Hashing

Problem / technique	Change to the shared idea
217. Contains Duplicate §3	Return whether len(set(nums)) differs from len(nums), or stop at the first repeated value in a scan.
242. Valid Anagram §3	Compare Counter(s) with Counter(t). Character order is irrelevant; multiplicity is the entire state.
1. Two Sum §3	Map each earlier value to its index. Look up target - x before storing x so one index cannot pair with itself.
49. Group Anagrams §3	Group words by their sorted-character string. A fixed-alphabet count tuple is the linear-per-word alternative when that bound is required.
347. Top K Frequent Elements §3	Count first, then place values in buckets indexed by frequency. Read buckets downward until k values are collected; this meets the better-than-sorting requirement.
238. Product of Array Except Self §4	Write the product strictly to the left into the output, then multiply by a running right product. No division; zeros need no special case.
36. Valid Sudoku §3	Check each filled digit in three seen sets: row r, column c, and box (r // 3, c // 3). Validate existing entries; do not solve the board.
271. Encode and Decode Strings §3	Write length, marker, payload. Decode the header, then jump by that length. Empty strings and markers inside payloads must round-trip.
128. Longest Consecutive Sequence §3	Put distinct values in a set. Walk a consecutive run only when its predecessor is absent. Starting at every element would repeat work.

25.2 Two Pointers

Problem / technique	Change to the shared idea
125. Valid Palindrome §5	Filter to lowercase alphanumeric characters and compare with the reverse. Use skipping end pointers only if constant extra space is required.
167. Two Sum II Input Array Is Sorted §5	Use sorted end pointers; move left when the sum is too small, right when too large. Return 1-based indexes and use distinct positions.

Problem / technique	Change to the shared idea
15. 3Sum §5	Sort, fix an anchor, and run pair search on the suffix. Skip duplicate anchors and duplicate matched endpoints; repeated values can still occupy distinct positions.
11. Container With Most Water §5	Measure width times the shorter wall. Move that wall because keeping it cannot improve the area as width shrinks.
42. Trapping Rain Water §4	Precompute the tallest wall from each side. Sum min(left_max, right_max) minus the local height. This direct linear-space version fits the base task.

25.3 Sliding Window

Problem / technique	Change to the shared idea
121. Best Time to Buy And Sell Stock §20	Treat each day as a possible sale. Store the minimum price so far and the best nonnegative profit. The purchase must precede the sale.
3. Longest Substring Without Repeating Characters §6	Keep a set of the current characters. Shrink until the incoming character is absent, then add it and measure the valid window.
424. Longest Repeating Character Replacement §6	Count the current uppercase letters. Shrink while window length minus the exact largest count exceeds k. Recomputing over 26 keys keeps the rule simple.
567. Permutation In String §6	Use a fixed window of len(s1). Add its new letter, remove its old letter, and compare 26-element count arrays. Equal counts mean a permutation.
76. Minimum Window Substring §6	Track missing required copies. Expand until none are missing, then shrink while valid. Store the best range and slice only once.
239. Sliding Window Maximum §6	Keep decreasing-value candidate indexes in a deque. Remove expired indexes at the front and dominated candidates at the back. Read the maximum at the front.

25.4 Stack

Problem / technique	Change to the shared idea
20. Valid Parentheses §7	Push openers. Each closer must match the top opener. Reject a missing or mismatched top, and require an empty stack at the end.
155. Min Stack §7	Push (value, minimum_so_far) pairs. Pop removes both facts together. Read the two fields of the top for top and getMin; all operations are constant time.

Problem / technique	Change to the shared idea
150. Evaluate Reverse Polish Notation §7	Push numbers; an operator pops right operand then left operand. Apply the operator and push its result. Division truncates toward zero, including negatives.
22. Generate Parentheses §13	Build a prefix with counts of opens and closes. Open while opens < n; close while closes < opens. Save complete strings of length 2*n.
739. Daily Temperatures §7	Store waiting day indexes. A strictly warmer incoming day resolves and pops colder days. Write index differences; equal temperatures must keep waiting.
853. Car Fleet §7	Sort by position descending. A car whose solo arrival time is no greater than the fleet ahead joins it; a larger time starts a new fleet.
84. Largest Rectangle In Histogram §7	Keep (earliest_start, height) pairs. A shorter bar finalizes taller rectangles. Carry the earliest popped start into the new bar and flush with a trailing zero.

25.5 Binary Search

Problem / technique	Change to the shared idea
704. Binary Search §8	Find the first index with value >= target. Check both bounds and equality; otherwise return -1. The required search time is logarithmic.
74. Search a 2D Matrix §8	Treat the row-ordered matrix as a virtual sorted array. Map a flat midpoint with divmod(mid, column_count); do not construct a flattened copy.
875. Koko Eating Bananas §8	Binary-search the first speed with sum((pile + speed - 1) // speed) <= h. Increasing speed never increases required hours.
153. Find Minimum In Rotated Sorted Array §8	Find the first index where nums[i] <= nums[-1]. Distinct rotated values make this predicate false then true, even when the array is unrotated.
33. Search In Rotated Sorted Array §8	Find the minimum pivot, choose the sorted side whose value range can contain target, then boundary-search that side. Distinctness is part of the proof.
981. Time Based Key Value Store §8	Keep parallel timestamp and value lists per key. Increasing set timestamps allow append. Get uses bisect_right(times, t) - 1; a negative index means no value.
4. Median of Two Sorted Arrays §8	Binary-search the left-half count taken from the shorter array. Adjust until both cross-cut comparisons hold. Use infinities at empty sides; the task requires logarithmic time.

25.6 Linked List

Problem / technique	Change to the shared idea
206. Reverse Linked List §9	Save next, point current.next backward, then advance previous and current. Return the final previous node, including None for an empty input.
21. Merge Two Sorted Lists §9	Build after a dummy tail by splicing the smaller current head. Advance that list and the tail. Attach the remaining list once the other ends.
143. Reorder List §9	Collect node references, then relink front, back, front, back. Terminate the final next link. For constant space, split, reverse the second half, and weave.
19. Remove Nth Node From End of List §9	Keep slow at a dummy and advance fast n nodes from head. Move both until fast ends; slow is now the target's predecessor. Bypass its next link.
138. Copy List With Random Pointer §9	First create a map from every original node to a fresh node, including None. Then translate next and random through that map. Values are not identities.
2. Add Two Numbers §22	Walk both digit lists with a carry. Emit total % 10 and keep total // 10 while either list or carry remains. Build after a dummy.
141. Linked List Cycle §9	Move slow one link and fast two. Meeting means a cycle; reaching an absent fast link means no cycle. Compare node identity, not values.
287. Find The Duplicate Number §9	Read index -> nums[index] as a functional graph. Floyd finds a meeting; restarting one pointer at index 0 locates the cycle entrance, the duplicate. No mutation or extra set.
146. LRU Cache §9	Keep keys in OrderedDict recency order. Move a key to the recent end on get and put; evict the oldest on overflow. If implementing internals, combine a map with a doubly linked list.
23. Merge K Sorted Lists §12	Put one front node from each sorted list in a heap. Pop, append, and replace it with its successor. Include a list identifier to break equal-value ties safely.
25. Reverse Nodes In K Group §9	Probe k nodes before changing links. Reverse that complete group with previous initialized to the following node, splice it back, and leave an incomplete suffix unchanged.

25.7 Trees

Problem / technique	Change to the shared idea
226. Invert Binary Tree §10	Visit every real node and swap its left and right references. An empty child stays empty. A stack avoids depth limits if needed.
104. Maximum Depth of Binary Tree §10	Build one level frontier at a time and count levels. Alternatively carry depth in a stack. Avoid recursive depth assumptions for a long chain.

Problem / technique	Change to the shared idea
543. Diameter of Binary Tree §10	Process children before parents. Store node-count heights; the edge-count diameter through a node is left_height + right_height. Keep the largest candidate.
110. Balanced Binary Tree §10	Use the same bottom-up heights. Every node must have child heights differing by at most one. An explicit reversed parent-first order handles deep chains.
100. Same Tree §10	Compare node pairs. Two missing nodes match; one missing node or unequal values fails. Both corresponding child pairs must match.
572. Subtree of Another Tree §10	Try every source node as a subtree root and perform an exact paired shape/value comparison. The direct O(n*m) approach fits these limits; use stacks for deep inputs.
235. Lowest Common Ancestor of a Binary Search Tree §10	Walk left if both target values are smaller and right if both are larger. The first split, including equality with a target, is the answer.
102. Binary Tree Level Order Traversal §10	Store a list of one level. Save its values, then replace it with all nonempty children in left-to-right order.
199. Binary Tree Right Side View §10	Use level traversal and take the last value of each level. This is the rightmost node because the frontier remains in left-to-right order.
1448. Count Good Nodes In Binary Tree §10	Carry the maximum ancestor value with each stack entry. Count the node when its value is at least that maximum, then update the value for its children.
98. Validate Binary Search Tree §10	Carry strict lower and upper bounds from all ancestors. Narrow the upper bound for a left child and lower bound for a right child. Equality is invalid.
230. Kth Smallest Element In a Bst §10	Traverse inorder with a stack and stop at the kth pop. For many queries with updates, store subtree sizes to guide rank selection.
105. Construct Binary Tree From Preorder And Inorder Traversal §10	Map values to inorder indexes. Consume preorder roots through subtree jobs on inorder ranges. Schedule right before left on a stack so preorder is consumed correctly; avoid repeated slicing.
124. Binary Tree Maximum Path Sum §10	Combine bottom-up gains. Send one nonnegative child branch upward; test both branches plus the node as a complete path. Initialize the best for all-negative trees.
297. Serialize And Deserialize Binary Tree §10	BFS-serialize values and explicit null markers. Decode with a queue: each real node consumes its two child tokens. Retain shape, negative values, and the empty tree.

25.8 Tries

Problem / technique	Change to the shared idea
208. Implement Trie Prefix Tree §11	Store a character-child dictionary per node and a terminal marker. Search requires a terminal; startsWith only requires that the path exists.
211. Design Add And Search Words Data Structure §11	Follow a literal edge; for a dot, try every character child at that position. The pattern must finish at a terminal marker. Use the same trie as ordinary prefix search.
212. Word Search II §11	Build one trie for all words and DFS the grid through trie edges. Restore path marks, remove found terminal markers, and delete exhausted child edges to prune future searches.

25.9 Heap / Priority Queue

Problem / technique	Change to the shared idea
703. Kth Largest Element In a Stream §12	Retain a min-heap of the largest k values across initialization and add calls. Push, trim when size exceeds k, then return the root. Duplicates count separately.
1046. Last Stone Weight §12	Heapify negative weights. Pop the two heaviest, push a nonzero difference, and repeat. Return the last weight, or zero if all stones disappear.
973. K Closest Points to Origin §12	Sort by squared distance x*x + y*y and take the first k points. No square root is needed. This clear sorting solution fits the stated limits.
215. Kth Largest Element In An Array §12	Keep the largest k values in a min-heap. Its root is the kth largest, including duplicate occurrences. Each push may be followed by one trim.
621. Task Scheduler §12	Fill cooldown+1 slots per round with distinct tasks of largest remaining count. Reinsert updated counts only after the round; include idle time except after the last task.
355. Design Twitter §12	Store per-author timestamped tweets and followee sets. Merge one newest candidate per relevant author with a heap. After each pop, advance only that author; stop after ten.
295. Find Median From Data Stream §12	Keep lower values in a max-heap and upper values in a min-heap. Preserve order and a size difference of at most one. Read the middle root or average of roots.

25.10 Backtracking

Problem / technique	Change to the shared idea
78. Subsets §13	Start with the empty subset. For each new value, add a copy of every old subset extended by that value. Each subset either contains the new value or does not.
39. Combination Sum §13	Sort positive candidates and recurse from a start index. Recurse at the same index to allow reuse. Stop when remaining is zero or the next value is too large.
46. Permutations §13	Choose each remaining position as the next value. Recurse on the smaller pool and extended path. Save when the pool is empty; input values are distinct.
90. Subsets II §13	Sort and recurse from a start index, saving every path. Skip equal choices at the same depth and recurse at i+1, so each occurrence is used at most once.
40. Combination Sum II §13	Sort, skip equal same-depth choices, and recurse at i+1. This changes unlimited Combination Sum to one use per input occurrence without duplicate result combinations.
79. Word Search §13	State is (row, column, word_index) plus cells used by this path. Match, mark, explore four neighbors, and unmark. Another starting path may reuse those cells.
131. Palindrome Partitioning §13	Choose each next prefix that is a palindrome and recurse on the suffix. A path of pieces is complete at the string end; copy it into the result.
17. Letter Combinations of a Phone Number §13	Expand a list of prefixes once per digit, using that digit's letter choices. This is a Cartesian product. If an interface allows empty digits, return an empty result.
51. N Queens §13	Place one queen per row. Track occupied columns, r-c diagonals, and r+c diagonals. Undo all three marks and the board cell after each recursive branch.

25.11 Graphs

Problem / technique	Change to the shared idea
200. Number of Islands §14	Scan string-valued land cells. Each unseen '1' starts one flood. Sink or mark the whole four-neighbor component before scanning onward; count starts, not individual cells.
133. Clone Graph §14	Map each original node to its clone before visiting neighbors. Reuse existing clones when cycles return to a node. Recreate neighbor links between copies only.
695. Max Area of Island §14	Use integer-valued flood fill and count each component's cells. Return the largest size, or zero for all water. Mark before adding neighbors to avoid duplicate work.

Problem / technique	Change to the shared idea
417. Pacific Atlantic Water Flow §14	Reverse-flow from each ocean's border, allowing equal or higher neighbors. A cell reaching both reversed floods can drain to both oceans. Return their intersection.
130. Surrounded Regions §14	Flood border-connected 'O' cells as safe. Flip only unmarked 'O' cells, then restore safe cells. Border reachability decides capture; size and shape do not.
994. Rotting Oranges §14	Seed every rotten orange and count fresh ones. BFS one level per minute, marking fresh cells immediately. Return -1 if fresh remains after the queue empties.
286. Walls And Gates §14	Seed all gates at zero distance. BFS into only unfilled rooms and write parent distance + 1. Walls block movement; unreachable rooms retain INF.
207. Course Schedule §15	Build prerequisite -> course edges and unmet-prerequisite counts. Repeatedly take ready courses. Finishing all courses means no dependency cycle blocks them.
210. Course Schedule II §15	Use the same readiness process and save its removal order. Return the order only if it contains every course, otherwise return an empty list.
684. Redundant Connection §15	Union endpoints in input order. The extra cycle edge is the first edge whose endpoints already share a component. Use n+1 storage for 1-based labels.
323. Number of Connected Components In An Undirected Graph §14	Build undirected adjacency, then count flood starts over all n nodes. Isolated nodes each form a component. Union-find is useful when edges must be processed incrementally.
261. Graph Valid Tree §15	Require n-1 undirected edges, then flood from one node and require all n nodes reached. The edge count plus connectivity characterizes a tree here.
127. Word Ladder §14	BFS over one-letter transformations that remain in the word set. Remove on enqueue. Return the word-count distance, including both endpoints, or zero if unreachable.

25.12 Advanced Graphs

Problem / technique	Change to the shared idea
332. Reconstruct Itinerary §16	Consume outgoing tickets in lexical order and append an airport only after its exits are exhausted. Reverse completion order. Duplicate tickets are separate edges; validity is guaranteed.
1584. Min Cost to Connect All Points §16	Dense Prim stores each outside point's cheapest edge to the current tree. Repeatedly select the least cost and update Manhattan distances. An $O(n^2)$ scan avoids all-edge storage.
743. Network Delay Time §16	Dijkstra computes shortest arrival times along nonnegative directed edges. Return the largest final distance, or -1 if any node is unreachable from the source.

Problem / technique	Change to the shared idea
778. Swim In Rising Water §16	Use a heap path cost equal to the maximum height encountered, not a sum. Start with the origin's height. The destination's cheapest popped cost is the required water level.
269. Alien Dictionary §15	Adjacent sorted words give an edge only at their first differing letter. Reject a longer word before its prefix. Deduplicate edges, include isolated letters, then topologically sort.
787. Cheapest Flights Within K Stops §16	Run k+1 relaxation rounds for at most k stops. Read only the old distance layer and write into a copy. One distance per city without a move layer loses needed state.

25.13 1-D Dynamic Programming

Problem / technique	Change to the shared idea
70. Climbing Stairs §17	Count ways by the last move: ways[i] = ways[i-1] + ways[i-2], with one way to form an empty path. The answer at n counts sequences of moves.
746. Min Cost Climbing Stairs §17	Define minimum cost to reach each position, including the top at n. Arrive from either previous step plus its cost. Positions 0 and 1 start with zero cost.
198. House Robber §17	For each house, compare skipping it with taking its value plus the best result two houses back. Start with zero for empty prefixes.
213. House Robber II §17	Handle one house directly. Otherwise solve the line once without the first house and once without the last; take the larger result because the endpoints are neighbors.
5. Longest Palindromic Substring §19	Expand around every single-character and gap center. Record the longest matching range and slice once. Odd and even palindromes both need centers.
647. Palindromic Substrings §19	Use the same two center types, adding one per successful outward expansion. Equal text at different positions counts as different substrings.
91. Decode Ways §17	Count prefix decodings. Add from i-1 for a valid nonzero last digit and from i-2 for a last pair from 10 through 26. Leading zero is not a letter.
322. Coin Change §18	Let dp[amount] be minimum coin count. Try each possible last coin; start dp[0]=0 and other entries unreachable. Unlimited reuse is allowed; greedy is insufficient.
152. Maximum Product Subarray §17	Keep the largest and smallest products ending here. Update both from the old pair and the new value alone. A negative value swaps useful extremes; zero allows restart.
139. Word Break §17	Let possible[i] mean the prefix s[:i] can be split. Try a last dictionary word after any reachable prefix. Start possible[0]=True; words may repeat.

Problem / technique	Change to the shared idea
300. Longest Increasing Subsequence §17	Default: endpoint DP tries all earlier smaller values in $O(n^2)$, fitting n<=2500. For the logarithmic follow-up, retain minimum tails by length and replace with bisect_left.
416. Partition Equal Subset Sum §18	Reject an odd total. Track subset sums up to half the total, using only old sums to add the next item. Each input occurrence is used at most once.

25.14 Blind 75 addition

Problem / technique	Change to the shared idea
377. Combination Sum IV §18	Count ordered sequences by their last number: ways[t] = sum(ways[t-x]) for fitting positive x. Start ways[0]=1; total-first iteration distinguishes different orders.

25.15 2-D Dynamic Programming

Problem / technique	Change to the shared idea
62. Unique Paths §19	Fill a path-count grid from above and left. Initialize the first row and column to one. Only right and down moves are legal, so dependencies are acyclic.
1143. Longest Common Subsequence §19	For prefix pair (i,j), equal last characters extend the diagonal by one. Otherwise take the larger result from skipping either last character. Empty-prefix lengths are zero.
309. Best Time to Buy And Sell Stock With Cooldown §17	State includes day and whether a share is held. Compare skipping with buying or selling. A sale transitions to the free state two days ahead, enforcing cooldown.
518. Coin Change II §18	Count unordered coin combinations. Process coins outside and totals upward inside. This allows reuse of the current coin without recounting different coin orders.
494. Target Sum §18	Cache (index, running_sum). Add counts from choosing plus and minus for the next value. At the end, return whether the target is reached; zero still creates two choices.
97. Interleaving String §19	Reject a total-length mismatch. State (i,j) consumes i and j characters; the target position is i+j. Either matching source may supply the next character.
329. Longest Increasing Path In a Matrix §19	Sort cells from highest value to lowest. A cell's answer is one plus the largest answer of a strictly higher neighbor. Equal heights cannot connect; long paths need no recursion.

Problem / technique	Change to the shared idea
115. Distinct Subsequences §19	Always count ways that skip the source's last character; also count using it when it matches the target's last character. Forming an empty target has one way.
72. Edit Distance §19	Prefix DP compares insert, delete, and replace, with no cost for equal final characters. Empty-to-prefix cost is its length; return the full-prefix cell.
312. Burst Balloons §19	Use open intervals between fixed padded walls. Try each interior balloon as the last burst and add its two solved sides plus the wall product. Fill narrow intervals first.
10. Regular Expression Matching §19	Cache input and pattern positions. A starred unit either consumes nothing or consumes one matching character while staying on that unit. Both inputs must finish together.

25.16 Greedy

Problem / technique	Change to the shared idea
53. Maximum Subarray §17	Track the best nonempty sum ending here: restart at x or extend by x. Keep a separate global best initialized from the first value for all-negative input.
55. Jump Game §20	Keep the farthest reachable index. An index beyond it proves a gap; otherwise extend reach with i+nums[i]. Reaching an index also makes all earlier positions reachable.
45. Jump Game II §20	Treat a reachable index range as one BFS layer. Its farthest outgoing jump sets the next layer's end. Count layers until the destination is included; reachability is guaranteed.
134. Gas Station §20	Reject insufficient total fuel. Scan net fuel and restart after each negative running tank. Every start inside that failed segment is also impossible.
846. Hand of Straights §20	The smallest unused card must start a consecutive group. Consume one copy of each required value, or fail. Count duplicates and reject a nondivisible total first.
1899. Merge Triplets to Form Target Triplet §20	Discard any triple exceeding the target anywhere. Take coordinate-wise maxima over all remaining triples. Equality with the target means they can be merged to form it.
763. Partition Labels §20	Store each character's final index. Extend the current partition through all final indexes encountered, and cut at the earliest boundary that closes every character inside.
678. Valid Parenthesis String §20	Track the feasible range of unmatched openers. A star expands both ends, a closer shifts down, and an opener shifts up. Reject negative maximum; finish with zero feasible.

25.17 Intervals

Problem / technique	Change to the shared idea
57. Insert Interval §21	Use the sorted disjoint order: copy intervals before the new one, merge all overlaps, then append those after it. Touching endpoints overlap in this task.
56. Merge Intervals §21	Sort by start. Merge into the last output interval when start <= last_end; otherwise start a new output interval. Preserve the larger end for containment cases.
435. Non Overlapping Intervals §21	Sort by end and keep each interval that starts at or after the last kept end. Earliest finish leaves the most remaining room. Return total minus kept.
252. Meeting Rooms §21	Sort by start and compare adjacent meetings. Fail when a previous end exceeds the next start. Equality is allowed: a meeting may start when another ends.
253. Meeting Rooms II §21	Sweep start +1 and end -1 events. Process ends first on ties, then take the maximum active count. Return zero for no meetings if the interface allows it.
1851. Minimum Interval to Include Each Query §21	Sort queries; push intervals that have started into a length-priority heap. Pop expired tops and read the shortest valid length. Restore original query order, including duplicates.

25.18 Math & Geometry

Problem / technique	Change to the shared idea
48. Rotate Image §22	Transpose across the main diagonal, then reverse every row. Both operations are in place and realize (r,c) -> (c,n-1-r) for the required clockwise rotation.
54. Spiral Matrix §22	Peel top, right, bottom, and left edges while shrinking bounds. Recheck for remaining rows and columns before the reverse edges so single rows and columns are not duplicated.
73. Set Matrix Zeroes §22	First collect rows and columns with original zeros, then write. For the constant-space follow-up, reuse the first row/column as markers with saved boundary flags.
202. Happy Number §22	Repeatedly sum squared digits. A seen set detects a loop; stop successfully at 1. The same deterministic-state reasoning applies to other repeated transformations.
66. Plus One §22	Scan digits right to left. Turn 9 into 0 and carry; increment the first smaller digit and return. If every digit was 9, prepend 1.
50. Pow(x, n) §22	Compute one half-power and square it; multiply by x when the exponent is odd. For negative powers, invert x and negate the exponent. Depth is logarithmic.
43. Multiply Strings §22	Multiply digit pairs into place i+j in reversed digit arrays. Normalize carries from low places to high places, remove leading zeros, and join digits. Do not parse the whole operands as integers.

Problem / technique	Change to the shared idea
2013. Detect Squares §22	Count coordinate copies. For each possible diagonal of a nonzero axis-aligned square, look up the two forced corners and multiply the three stored multiplicities.

25.19 Bit Manipulation

Problem / technique	Change to the shared idea
136. Single Number §23	XOR all values. Every paired occurrence cancels, leaving the single value. The exact twice-except-once multiplicity is essential to this rule.
191. Number of 1 Bits §23	Use bit_count(), or repeatedly count the low bit and shift. The alternative n &= n-1 removes one set bit per step; apply these loops to nonnegative inputs.
338. Counting Bits §23	Build counts[i] = counts[i » 1] + (i & 1). The smaller number's answer is already available. Initialize counts[0]=0 for the required linear pass.
190. Reverse Bits §23	Read the low bit and append it to the result, then shift the input. Run exactly 32 rounds so leading zeros become trailing zeros correctly.
268. Missing Number §22	Subtract sum(nums) from n*(n+1)//2. Exactly one distinct value is missing from 0 through n; this gives linear time and constant extra space.
371. Sum of Two Integers §23	Iterate XOR for carry-free sum and shifted AND for carry, masking both to 32 bits. Convert the final sign bit to Python's signed integer interpretation.
7. Reverse Integer §22	Pop digits from the magnitude and push into the reversed result. Before result*10+digit, compare against a sign-specific 32-bit limit; return zero on overflow.

Sources and verification

The problem set was checked against NeetCode's published list data. The original Blind 75 author links Combination Sum IV in the original question collection; it is included here in addition to the current NeetCode 150.

Statements, examples, and constraints were reviewed individually through LeetCode's public problem descriptions. For seven Premium statements, the public NeetCode equivalents were used: Encode/Decode, Walls/Gates, Components, Valid Tree, Alien Dictionary, Meeting Rooms, and Meeting Rooms II. Their bounds and names can differ from LeetCode's. The selected methods cover the shared tasks and these published bounds. Reference solutions were checked in NeetCode's Python repository; the guide uses independently written examples and favors the simplest approach meeting the requirements.

Python behavior was checked against the official documentation for built-in types, collections, heapq, bisect, and functools. See `coverage.json` for per-problem sources and methods. From the repository root, run `python3 cheatsheet/check_examples.py` to check displayed examples. These checks cover the displayed code, not all judge suites.